MusicID: song recognition from first principles, measured and calibrated

Technical report · November 2026 · every number here is reproducible from the project folder (section 11)

Summary

MusicID began as a re-implementation of the Shazam landmark algorithm (Wang, 2003). This report turns it into a measured system. The original code is reproduced bit-for-bit, then evaluated on a 4,000-song library from the Free Music Archive with 17,600 test clips under ten recording conditions and five clip lengths, next to the open-source fingerprinter audfprint. The guessed match threshold is replaced by a statistically derived one with a stated false-match rate, and a neural fingerprint plus a learned decision layer are added.

- The original rule "score > 39" names a song for **21.4%** of clips that are *not* in the library (4,000 songs), a rate that grows with library size; it finds the right song for 53.9% of library clips.
- The improved classical engine (v2) with a derived threshold finds 81.2% at a false-match rate of 1.2% (target 1%), and searches 8× faster.
- The hybrid of classical and neural evidence finds **93.9%** of library clips at a false-match rate of 1.4% (95% CI 1.0%–1.8%); its top candidate is right 98.1% of the time.
- Plain Poisson statistics are badly over-confident because votes arrive in clumps; a quasi-Poisson correction (estimated ϕ = 1.5) and split-conformal thresholds fix this. Checking every second while recording is kept honest by splitting the error budget over the checks (Bonferroni).

Method	Correct matches	False matches (95% CI)	Top-1	Search time
Original MusicID (score > 39)	53.9%	21.4% (19.2%–23.9%)	53.9%	233 ms
audfprint, default threshold	50.5%	3.8% (3.2%–4.5%)	59.4%	–
audfprint, conformal 1%	42.2%	0.5% (0.3%–0.7%)	59.4%	–
v2 classical, quasi-Poisson p < 1%	80.4%	1.4% (1.0%–1.8%)	85.8%	30 ms
v2 classical, conformal 1%	81.2%	1.2% (0.9%–1.7%)	85.8%	30 ms
Neural fingerprint, conformal 1%	90.1%	1.6% (1.0%–2.1%)	98.0%	46 ms
Hybrid (classical + neural, learned), conformal 1%	93.9%	1.4% (1.0%–1.8%)	98.1%	v2 + neural

All test clips (main + length grids): 8,800 from library songs and 8,591 from songs not in the library. "Correct matches" = right song AND accepted by the decision rule. Search time = median database search per 5-s clip on a laptop CPU (fingerprinting excluded). CI = bootstrap over songs.

1. Where the method comes from

The fingerprint is the constellation-map and landmark-hashing method of Wang (2003). The original MusicID code also follows the widely read tutorial "How does Shazam work" (coding-geek.com, 2015), including its "address / couple" naming and its target zone of five points. None of that design is claimed as new here. What this project adds is the measurement, the statistics, the learned components and the engineering: an exact replica that runs at scale, a benchmark with an external baseline, a derived decision rule with a stated error rate, a neural fingerprint (after Chang et al., 2021) and a calibrated fusion of the two.

2. Step one: an exact, fast replica of the original

The original pipeline (AudioModule + DBModule) was re-implemented with vectorised NumPy and checked against the original code: on real recordings and on synthetic audio the constellation maps are **bit-identical**, the 52,755 stored hashes of the song database are identical, and the decision (song and score 1,308) is identical (tests/test_engine.py). The replica keeps every quirk so that the original algorithm is what gets evaluated:

- stereo is averaged and mono input halved; the signal is zero-padded to a power of two before filtering;
- a 40,001-tap windowed-sinc low-pass filter (5 kHz, 4.4 Hz transition band) followed by averaging blocks of four samples;
- windowed frames are stored as 16-bit floats before the FFT (a side effect of numpy.insert into a float16 array);
- the peak threshold divides by 6 although 5 bands are used, and each peak time has an extra j/11025 s term (j = frequency bin);
- target zone = "the next five points of the list", time differences in rounded milliseconds.

The hand-written recursive FFT matches numpy.fft to 10^{-9} (unit test), so the experiments use NumPy's FFT for speed.

3. The improved classical engine (v2)

v2 keeps the landmark idea but follows Wang (2003) more closely. Each change addresses something the replica revealed:

- **Anti-aliasing:** polyphase resampling to 11,025 Hz from any input rate (the original only works at 44.1 kHz).
- **Frames:** 50% overlapping Hann windows of 1,024 samples (hop 46.4 ms), so a clip that starts between frames still lines up.
- **Peaks:** local maxima of the log spectrogram in a 7-frame × 21-bin neighbourhood, at most 30 per second (215 Hz–5.4 kHz).
- **Target zones:** each anchor is paired with up to 6 later peaks 1–40 frames ahead and within ±100 bins, instead of "the next five list items".
- **Hash:** 24 bits = anchor bin (9) | target bin (9) | Δt in frames (6); times are integer frames, so offsets never split by rounding.
- **Index:** three sorted arrays (key, song, time) searched by bisection instead of a dictionary of strings; 19.2 M hashes for 4,000 songs in 293 MB.

Votes. A query hash that matches a database hash of song *s* votes for the time offset $\delta = t_{db} - t_{query}$. Each query anchor votes at most once per (song, offset), and a song's score is the largest number of votes in a window of three neighbouring offsets (δ−1, δ, δ+1), which absorbs one-frame misalignment.

4. Statistics: a threshold with a known error rate

4.1 A null model for the votes

If the clip is not from song *s*, its M_s votes for *s* land on the K_s possible offsets (song length + clip length, in frames) at random, so the count in one window of width w = 3 is approximately Poisson with mean $w\lambda_s$, $\lambda_s = M_s/K_s$. The best score must beat the maximum over all offsets of all songs, which gives the family-wise p-value of an observed top score c:

$$p(c) \;=\; 1 - \prod_s \Pr\left(\max_\delta N_{s,\delta} < c\right) \;\approx\; 1 - \exp\left(-\sum_s K_s \Pr\left[\mathrm{Poisson}(w\lambda_s) \geq c\right]\right)$$

A match is declared when p(c) < α (α = 1%). Unlike a fixed score, this threshold rises automatically with the number of songs, the number of votes and the clip length.

4.2 The plain model fails, and why

On held-out non-library clips the plain Poisson rule gives far more false matches than it promises: 13.0% instead of 1% for v2, and 88.6% on the original fingerprint. Votes are not independent: one coincidental alignment of a repeated musical pattern produces a clump of votes at the same offset, so counts are over-dispersed (variance > mean). The original hashing is extreme: even with a dispersion factor of 4 its false-match rate is still 10.9%. The statistics only work on top of a hash design whose votes are close to independent.

4.3 Two fixes

Quasi-Poisson. Treat score/ϕ as Poisson with mean $w\lambda_s/\phi$ and choose the smallest ϕ on a grid whose false-match rate on calibration clips is at most α; this gives ϕ = 1.5 for v2.

$$p_\phi(c) \;\approx\; 1 - \exp\left(-\sum_s K_s \Pr\left[\mathrm{Poisson}(w\lambda_s/\phi) \geq \lceil c/\phi \rceil\right]\right)$$

Split conformal. For any score where larger means "more match-like" ($-\log_{10}p$, the neural similarity, or the fusion log-odds), take n calibration clips known not to be in the library, sort their scores, and use the k-th smallest as threshold. If test non-library clips are exchangeable with the calibration ones, P(false match) ≤ α with no distributional assumption. Thresholds are set separately for each clip length (Mondrian conformal), because a 1-s clip has a much wider null distribution than a 10-s clip.

$$\hat{\tau} = s_{(k)}, \quad k = \lceil (1-\alpha)(n+1) \rceil \qquad \Rightarrow \qquad \Pr\left(S_{\mathrm{new}} > \hat{\tau} \,|\, H_0\right) \leq \alpha$$

Honest uncertainty. Test clips come from 490 non-library songs, about 17 clips each, and a few songs that resemble a library song produce many false matches (up to 13 each). The effective sample size is the number of songs, so every confidence interval bootstraps over songs, not clips.

4.4 Checking every second: sequential testing

The app re-checks the growing recording every second. Testing T times at level α each inflates the overall false-match rate ("peeking"). Splitting the budget, α/T per look (Bonferroni), keeps the overall rate at most α:

$$\Pr\left(\exists\, t \leq T\colon\; p_t < \alpha/T \,|\, H_0\right) \;\leq\; \sum_{t=1}^{T} \alpha/T = \alpha$$

5. Machine learning

5.1 A neural fingerprint learned by contrastive learning

Each 1-s segment (64-band log-mel spectrogram, 150 Hz–5 kHz, mean-normalised for gain invariance) is mapped by a small CNN (five 3×3 convolution layers, 0.67 M parameters) to a 128-dimensional unit vector f(x). Training pairs are a clean segment and a degraded copy of the same moment: time shift up to ±0.3 s, pink noise at 0–20 dB, background music, room

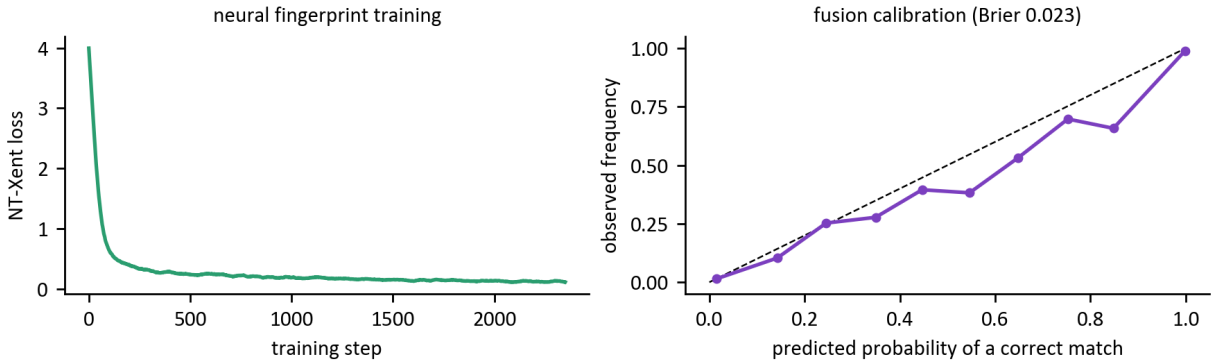
echo (RT60 0.2–0.8 s), random EQ, ±3% speed change and gain. The NT-Xent loss (temperature τ = 0.05) pulls each pair together and pushes away the other 2B−2 segments of the batch:

$$\ell_i = -\log \frac{\exp\left(f(x_i)^\top f(x_i^+)/\tau\right)}{\sum_{j \neq i} \exp\left(f(x_i)^\top f(x_j)/\tau\right)}$$

It was trained on 2,992 songs that are *not* in the evaluation library (2,400 steps, batch 192, ~45 minutes on a laptop CPU). The library is stored as 228,000 vectors (one per 0.5 s). A query is cut into 1-s segments every 0.5 s; each finds its 20 nearest library segments, and votes are aggregated per (song, offset) exactly as in the classical method; the score is the mean best similarity of the agreeing segments.

5.2 A learned, calibrated decision layer (hybrid)

Every clip has two candidates: the classical top song and the neural top song. Two classifiers estimate P(candidate is right) from 13 features (classical −log p, scores and their ratio, votes, hashes, original-engine p, neural best and second similarity and margin, whether the engines agree, log clip length). Logistic regression beat gradient boosting in 5-fold cross-validated log-loss. Training uses the calibration set (2,000 library + 1,000 non-library clips) weighted 50/50; the final threshold is split-conformal on 2,500 separate non-library clips. Probabilities are well calibrated on the test set (Brier score 0.023).



Left: contrastive training loss. Right: reliability of the fusion probabilities on test clips (points near the diagonal = honest probabilities).

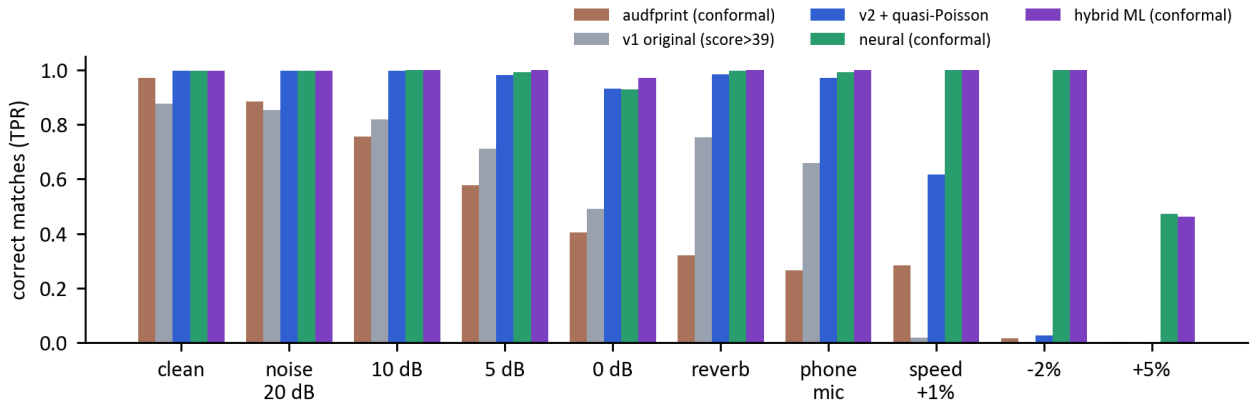
6. Benchmark

Data. FMA-small (Defferrard et al., 2017): 8,000 Creative-Commons tracks of 30 s, of which 7,992 decode correctly. Fixed random split: 4,000 library songs, 1,000 non-library songs (500 for calibration, 500 for testing), 2,992 for training the neural model.

Clips. Random start positions; lengths 1, 2, 3, 5 and 10 s; conditions: clean, pink noise at 20/10/5/0 dB SNR, room reverb (RT60 0.6 s), a phone-in-a-room simulation (150 Hz–5 kHz band-pass, RT60 0.4 s, noise at 10 dB), and playback speed +1%, −2%, +5%. Test grids: all ten conditions at 5 s, and all lengths for clean / 5 dB / phone mic; 400 library and 400 non-library clips per cell. Every engine sees exactly the same audio.

Duplicates. FMA contains the same music under several ids (re-uploads, vocal and instrumental versions). Tracks with the same artist and the same base title are grouped by metadata only; 355 non-library clips whose group exists in the library were excluded from false-match counts, and naming any member of the right group counts as correct.

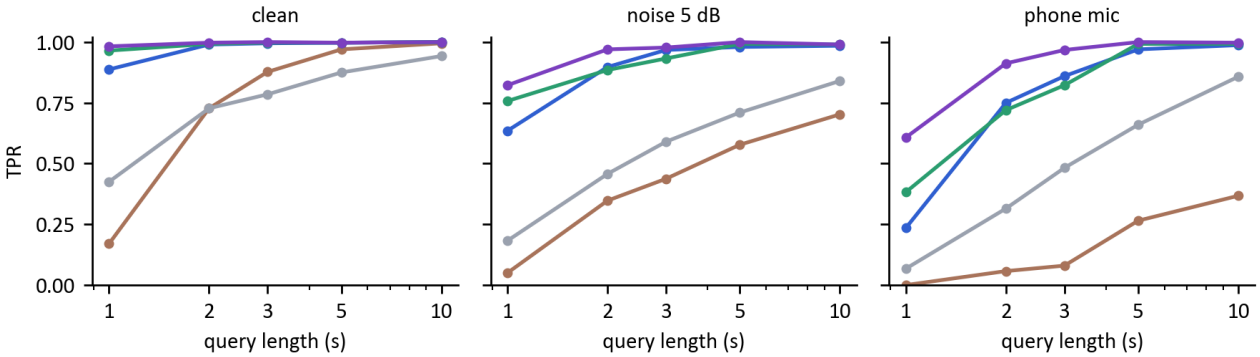
7. Results



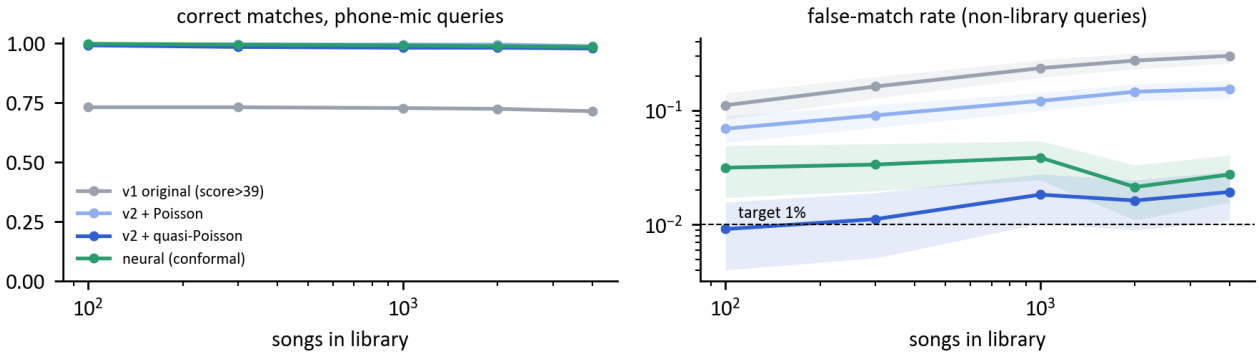
Correct matches for 5-s clips at a 1% false-match target (the original uses its fixed rule). v2 and the neural model are both near-perfect up to 5 dB noise; small speed changes break every landmark method, while the neural model (trained with ±3% speed) tolerates ±2%. The hybrid takes the best of both.

Condition (5 s)	Original	audfprint	v2 (conformal)	Neural	Hybrid
clean	88% / 27.6%	97% / 0.5%	100% / 2.6%	100% / 2.3%	100% / 1.5%
noise 20 dB	85% / 33.4%	88% / 1.0%	100% / 1.5%	100% / 2.6%	100% / 2.0%
noise 10 dB	82% / 30.2%	76% / 0.5%	100% / 1.8%	100% / 2.5%	100% / 2.8%
noise 5 dB	71% / 19.3%	58% / 1.0%	98% / 2.5%	99% / 2.8%	100% / 3.1%
noise 0 dB	49% / 10.7%	40% / 0.5%	93% / 1.8%	93% / 1.3%	97% / 1.8%
reverb	75% / 29.5%	32% / 0.0%	98% / 1.5%	100% / 2.1%	100% / 2.3%
phone mic	66% / 31.1%	26% / 0.3%	97% / 1.5%	99% / 2.8%	100% / 2.3%
speed +1%	2% / 23.5%	28% / 1.0%	62% / 0.5%	100% / 2.0%	100% / 2.0%
speed −2%	0% / 14.1%	2% / 0.8%	3% / 1.5%	100% / 2.3%	100% / 2.8%
speed +5%	0% / 19.7%	0% / 0.8%	0% / 0.5%	47% / 1.3%	46% / 1.0%

Each cell: correct matches / false-match rate (400 + 400 clips).



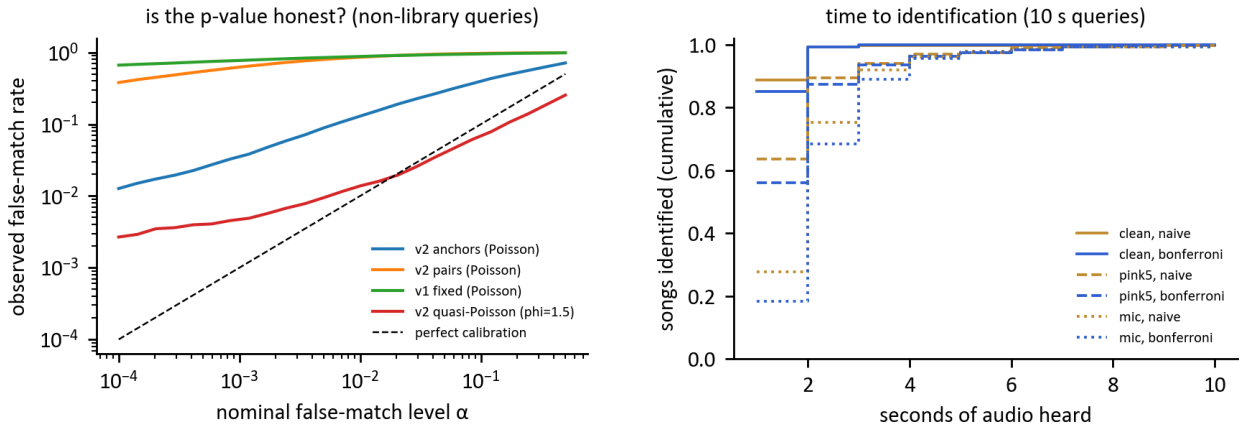
Correct matches vs clip length. The hybrid identifies 98% of clean 1-s clips and 61% of 1-s phone-mic clips (original: 42% and 7%).



Library size. The fixed rule's false-match rate climbs with every song added; the derived threshold stays close to the 1% target (shaded: 95% CI, bootstrap over songs).

Songs in library	100	300	1,000	2,000	4,000
Original (> 39): false matches	11.1%	16.2%	23.4%	27.3%	30.0%
v2, plain Poisson: false matches	6.9%	9.1%	12.1%	14.6%	15.5%
v2, quasi-Poisson: false matches	0.9%	1.1%	1.8%	1.6%	1.9%
Original: correct, phone mic	73.0%	73.0%	72.7%	72.3%	71.3%
v2, quasi-Poisson: correct, phone mic	99.0%	98.3%	98.0%	98.0%	97.7%

Calibration and sequential decisions



Left: observed vs promised false-match rate on non-library clips. Plain Poisson (blue, orange, green) promises far more than it delivers; quasi-Poisson (red) follows the diagonal near α = 1%. Right: how quickly 10-s recordings are identified when checked every second (v2, quasi-Poisson).

Recording	Rule	Correct	False matches	Mean time
clean	every second, α each	99.7%	8.7%	1.1 s
clean	every second, α/10 each	100.0%	2.3%	1.2 s
clean	once, after 10 s	100.0%	4.7%	10.0 s
noise 5 dB	every second, α each	99.7%	5.9%	1.6 s
noise 5 dB	every second, α/10 each	100.0%	1.6%	1.7 s
noise 5 dB	once, after 10 s	100.0%	3.3%	10.0 s
phone mic	every second, α each	99.3%	6.2%	2.1 s
phone mic	every second, α/10 each	99.3%	1.2%	2.3 s
phone mic	once, after 10 s	99.3%	2.9%	10.0 s

Checking every second at level α each ("peeking") multiplies the false-match rate; splitting α over the looks brings it back down at a cost of about 0.1–0.2 s.

8. Was the 40,001-tap filter necessary?

Aliases only hurt if they land on frequencies the fingerprint uses (up to 5.5 kHz). After decimation to 11,025 Hz, content at f folds to 11,025 − f, so only content above 6,025 Hz can land below 5 kHz: a filter that passes 5 kHz and stops 6,025 Hz is enough. A Kaiser-window design for that requirement (60 dB) needs **157 taps** instead of 40,001. Everything else held fixed (original fingerprint with its bugs fixed, 1,000-song library):

Anti-aliasing	Taps	Time / 29-s song	Peaks shared with original	Correct (clean / 10 dB / mic)	False matches
40,001-tap sinc + average of 4 (original)	40001	851 ms	100%	99% / 95% / 89%	93.0% / 93.7% / 96.3%
Kaiser FIR + average of 4	157	865 ms	98.5%	99% / 96% / 91%	95.0% / 92.0% / 96.3%
polyphase resampling (scipy)	default	99 ms	92.3%	100% / 96% / 91%	94.7% / 91.0% / 96.0%

Peaks shared = identical (time, frequency) peaks after removing each filter's delay (100 songs). Decisions use the plain Poisson rule, so false-match rates are high for all three; the comparison is between filters.

Two side findings. (1) The original filter delays the audio by 20,000 samples (0.45 s) because its output is never re-centred; this is harmless for matching, since songs and clips are delayed equally, but every time stamp is shifted. (2) Filter length barely changes run time here, because FFT convolution costs about the same for 157 or 40,001 taps on a padded signal; polyphase resampling, which never computes the samples it throws away, is 8.6× faster end to end. Answer for an interviewer: the requirement follows from where aliases fold to, and a filter sized from it (157 taps) gives the same

fingerprint.

9. What did not work, and what was learned

- **Fixing the original's bugs changed almost nothing** (correct matches 71.1% vs 71.0%). The real problems were the hash design and the decision rule, not the ÷6 or the j/11025 term.
- **The textbook Poisson model is wrong for real music.** It only becomes usable after an over-dispersion correction, and only on top of a hash design with near-independent votes.
- **Landmark hashing cannot handle speed changes:** at +1% v2 drops to 62.5% and at −2% to 3.0%; the neural model, trained with speed augmentation, keeps 100% at −2% but also fails at +5%, outside its training range.
- **The neural model is weaker than v2 on very noisy, very short clips;** the learned fusion exploits that the two fail on different clips.
- **Benchmarks hide traps:** FMA duplicates looked like false matches until they were traced by metadata, and per-song clustering makes naive confidence intervals far too narrow.
- **Conformal guarantees hold on average over calibration sets.** With ~490 calibration songs per length, the realised rate can land somewhat above 1%, which is what the intervals show.

10. Limitations and next steps

- Degradations are simulated (noise, synthetic room responses); only one live microphone test was done. A recorded test set (phones in real rooms) is the most important next step.
- Library songs are 30-s clips and the library has 4,000 songs; commercial systems index millions of full songs. Memory per song (~4,800 hashes) and the calibration would need re-checking at that scale.
- The neural model is small and CPU-trained; a larger model trained longer, and pitch/tempo-invariant hashes (e.g. Panako-style triplets), are natural extensions.
- The quasi-Poisson factor is one global constant; modelling the dispersion as a function of clip length and song would be sharper.

11. Reproducing everything

From the project folder, in order: *python research/download_fma.py* (7.7 GB) · *python -c "from research import data; data.build_cache()"* · *python -m research.bench build*, then *prepare*, *patch_v1*, *classical* · *python -m research.train_neural* · *python -m research.bench neural*, then *conformal*, *rerun_seq* · *python -m research.baseline_audfprint* · *python -m research.filter_study* · *python -m research.analyze* · *python -m research.make_report*. Unit tests: *python -m pytest tests*. All random choices are seeded.

References

A. Wang (2003). An industrial-strength audio search algorithm. Proc. ISMIR.
coding-geek.com (2015). How does Shazam work. Online tutorial.
S. Chang et al. (2021). Neural audio fingerprint for high-specific audio retrieval based on contrastive learning. Proc. ICASSP.
M. Defferrard, K. Benzi, P. Vandergheynst, X. Bresson (2017). FMA: a dataset for music analysis. Proc. ISMIR.
D. Ellis. audfprint: landmark-based audio fingerprinting. github.com/dpwe/audfprint.
V. Vovk, A. Gammerman, G. Shafer (2005). Algorithmic Learning in a Random World. Springer.
A. Angelopoulos, S. Bates (2023). Conformal prediction: a gentle introduction. Foundations and Trends in Machine Learning.
T. Chen, S. Kornblith, M. Norouzi, G. Hinton (2020). A simple framework for contrastive learning of visual representations. Proc. ICML.
J. Six, M. Leman (2014). Panako: a scalable acoustic fingerprinting system handling time-scale and pitch modification. Proc. ISMIR.